

ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, performance, limitations)	1
TL;DR	1
Selecting and confirming the solver	2
Performance — the default is not the fast one	3
Noise and pole-zero under KLU (Enhancement-113)	3
KLU discrepancies — all resolved	4
Convergence aids and later solver fixes	5
The dual-solver test harness	5

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, performance, limitations)

This build of ngspice-46 ships two direct linear solvers. They now agree everywhere on *results*, but they are very far apart on *cost*. This note records exactly how they differ — which is the default, how to select and confirm each, what each costs, and where KLU falls short — based on a direct read of the source, a solver-by-solver sweep of the whole [examples/](#) suite, and a measured scaling benchmark.

The headline, if you read nothing else: the default (Sparse 1.3) is the *correct* one, not the *fast* one. On Verilog-A/OSDI circuits its runtime grows steeply superlinearly while KLU's stays roughly linear — a measured 68× at 380 device instances, for an identical answer. See [Performance](#).

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 45 <code>klu_*</code> symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	<code>.option klu</code>	<code>.option sparse</code> (or just the default)
Cost on large circuits	✓ ~linear in circuit size	✗ steeply superlinear — 68× slower at 380 OSDI instances (below)
DC op / DC sweep	✓ correct	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (<code>.noise</code>)	✓ correct (since E-113)	✓ correct
Pole-zero (<code>.pz</code>)	✓ correct (single-ended E-113; balanced-output + complex-root determinant E-171/172)	✓ correct
Sensitivity (<code>.sens</code> , DC & AC)	✓ correct (since E-114)	✓ correct

	KLU	Sparse 1.3
Distortion (.disto)	✓ correct (since E-115)	✓ correct
Periodic steady state (.pss)	✓ correct (since E-118)	✓ correct
Periodic small-signal (.pac/.pnoise/.pxf/.psp)	✓ correct	✓ correct
Transient checkpoint/ restart (savestate/loadstate)	✓ correct (since E-180; incl. cross-solver restore)	✓ correct

The periodic small-signal analyses (PAC / Pnoise / PXF and the E-132 periodic S-parameters **.psp**) build a dense $(2M+1)N$ harmonic conversion matrix and solve it with a standalone dense complex LU (**pss_solve**) that is independent of the sparse linear solver — so they inherit the solver only through the underlying PSS, which runs correctly under both since E-118 (KLU forces a full re-factor every shooting step, so it pays a small premium — dramatic before [Enhancement-176](#), whose driven-mode shooting cut a pumped run from millions of breakpoint-flooded timepoints to a few hundred, making PSS cheap under both solvers). The E-133 two-tone **qpss** and the E-134 frequency-domain **hb** (harmonic balance) are likewise solver-independent — **qpss** drives a transient and direct-DFTs it; **hb** does a dense complex Newton on the conversion matrix and uses the sparse solver only to *read* the periodic $G(t)/C(t)$ off the device matrix. That read is the one solver-specific detail: Sparse uses **spSetComplex**, KLU needs the complex-CSC binding (**DEVbindCSCComplex**), exactly as the PAC harmonic extraction does — **hb** carries the same **#ifdef KLU** branch, so it is verified bit-identical under **.option klu** and **.option sparse** (a bare **hb** also copies the task's KLU mode before building the matrix, so **.option klu** takes effect without a prior analysis). Transient checkpoint/restart ([Enhancement-131](#)) was guarded Sparse-only until [Enhancement-180](#) diagnosed the real defect: the E-131-era explanation ("KLU factorization objects absent on the restore path") was wrong — **loadstate** called **CKTsetup** *before* the analysis dispatch copies the task's **.option klu** into the circuit, so the matrix was built as Sparse; the resume dispatch then flipped the circuit to KLU mode over it and **NIiter** dereferenced the **NULL SMPkluMatrix**. Copying the task's solver selection (and the E-152 KLU knobs) into the circuit before setup fixes it: all four **save×load** solver combinations — including cross-solver restores (the checkpoint file contains only solver-agnostic state) — now continue exactly on the uninterrupted run's trajectory.

Practical guidance: on a small circuit, leave the default — the two are close there ($1.2\text{--}1.4\times$ across 19–38 device instances) and Sparse 1.3 is the one every result here is keyed off. On anything large — roughly a hundred device instances and up — reach for **.option klu**, and expect the gap to widen with every device you add (**Performance** measures $4\times$ at 95 instances and $68\times$ at 380). This advice used to read "leave the default unless you have a specific reason to switch"; the measurement below is that reason, and the reason it now says otherwise. Correctness is not the deciding factor any more — both solvers agree across the entire suite. Since [Enhancement-113](#) KLU also runs noise and single-ended pole-zero correctly, and since [Enhancement-114](#) it runs DC/AC sensitivity, and since [Enhancement-115](#) distortion (**.disto**) correctly; since [Enhancement-171/172](#) the pole-zero path is fully KLU-correct too (complex-plane determinant, balanced output, full-partial-pivot fallback) — no analysis is Sparse-only under KLU any more, and since [Enhancement-180](#) no *feature* is either (transient checkpoint/restart, the last one, now runs — and even restores across solvers).

The one thing to keep in mind when you switch: KLU's symbolic ordering is computed once and cannot re-pivot dynamically the way Sparse does, so it is in principle less forgiving of a near-singular Jacobian on a stiff transient edge. In practice the only case that ever exhibited this — **opamp741**'s slew — turned out to be caused by the *trapezoidal* integrator ringing, not by KLU, and the dissipative Gear integrator removes it (see the **former opamp741 discrepancy** below). **.option method=gear** is the right companion for a stiff transient under either solver.

Tuning KLU ([Enhancement-152](#)): KLU's reordering and scaling, previously hard-coded, are now **.options** — **klu_ordering=amd|colamd** (fill-reducing ordering), **klu_scale=none|sum|max** (row equilibration), and **klu_btf=on|off** (block-triangular-form permutation), with defaults **amd/max/on**. They change only *how* the matrix factors, never the solution (verified physically identical to $\sim 1e\text{--}14$ across all settings), and matter only on unusual matrix structures (a different ordering can reduce fill) or badly-scaled matrices. The **klu_memgrow_factor** knob was also fixed (it had silently collapsed to a boolean).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu      * use KLU (SuiteSparse)
.option sparse   * use the legacy Sparse 1.3 (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive option command, which prints the live CKTkluMODE:

```
.control
run
option      * prints, among other things: "Matrix solver: KLU" or "Sparse 1.3"
.endc
```

On the committed bin/ binary this reports **Sparse 1.3** for a bare deck and for `.option sparse`, and **KLU** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Performance — the default is not the fast one

Everything above is about *correctness*, where the two solvers now agree everywhere. Cost is a different story, and on the circuits this project exists to serve — Verilog-A compact models through OSDI — it is not a close call.

Benchmark. `bjt741.va` (the ~70-line Verilog-A BJT of the [opamp741 example](#)) instantiated as N copies of the 20-transistor μ A741 follower — 19 OSDI instances per opamp — run as `tran 10n 40u` under `.option method=gear`, on the committed binary. Both solvers produce the same answer: identical timepoint counts (4014) and a worst-case deviation of 3.8e-07 across all outputs, so this compares like with like.

741s	OSDI instances	Sparse 1.3 (default)	KLU	ratio
1	19	0.05 s	0.04 s	1.2×
2	38	0.10 s	0.07 s	1.4×
5	95	0.65 s	0.16 s	4.1×
10	190	8.74 s	0.30 s	29×
20	380	40.2 s	0.59 s	68×

KLU is essentially linear in circuit size — each doubling roughly doubles its runtime. Sparse 1.3 is steeply superlinear: each doubling multiplies its runtime by 4.6–13×. The crossover is early, around a hundred instances, and there is no size at which Sparse wins.

It is not an artifact of the benchmark's structure. N independent opamps form a block-diagonal matrix, which is the ideal case for KLU's block-triangular-form permutation, so the same sweep was repeated with the opamps wired into a connected cascade (each stage buffering the previous one's output): 63.7× at 380 instances, essentially unchanged. The gap is a property of the solvers, not of the topology.

Where the time goes. Sampling the 380-instance run under the default solver:

symbol	share of runtime
spFactor (Sparse numeric factorization)	88.4%
linear solve, total	99.1%
OSDI model eval()	0.3%

Two things follow. First, the cost is entirely in the factorization, not in evaluating the device models — so this is a solver problem, and no amount of model- side work (device bypass, latency exploitation, faster eval) can address it: the whole of eval is 0.3%. Second, ngspice's inability to handle very large post-layout netlists — listed as a gap against commercial simulators in [ngspice_gaps.md](#) — is substantially a default-solver artifact rather than an inherent limit. `.option klu` moves that wall a long way out.

Caveat on scope. These numbers are one model family (a bipolar compact model) in transient. The *direction* is not in doubt — it follows from Sparse 1.3's linked-list structures and Markowitz re-pivoting on every factorization versus KLU's fixed symbolic ordering and cache-friendly compressed-column refactor — but the exact ratio on a given circuit will differ. Measure your own deck; the two solvers agree on the answer, so switching costs nothing but the flag.

Noise and pole-zero under KLU (Enhancement-113)

ngspice used to refuse both outright (`noisean.c` / `pzan.c` printed "not (yet) supported with 'option KLU'"). The real reason for noise was subtler than "not wired up": noise uses the adjoint method — it solves the *transposed* system

$A^T \cdot x = e$ via `SMPcaSolve` — and `SMPcaSolve`'s KLU branch was calling the non-transposed `klu_z_solve` where its Sparse branch calls `spSolveTransposed`. That silently produced the wrong noise for any asymmetric matrix (every circuit with a transistor or controlled source), which is why it was disabled rather than merely slow.

[Enhancement-113](#) fixes the adjoint solve (`klu_z_solve`) and lifts the guards, so under KLU:

- Noise is correct — verified identical to Sparse on resistor thermal noise, asymmetric VCCS circuits, OSDI device models, and integrated totals. (`.sp` S-parameters share the same adjoint solve and are corrected too.)
- Single-ended pole-zero (grounded output reference) runs correctly.
- Balanced-output pole-zero (non-grounded reference) was Sparse-only at first: its zeros phase folds columns at solve time, which Sparse survives via dynamic Markowitz re-ordering but KLU's fixed symbolic factorization could not. [Enhancement-172](#) closed it — `CKTpzSetup` reserves the union pattern before `COO→CSC` conversion and `SMPcAddCol` gained a merge-walk KLU branch — and replaced the out-of-range pivot-tolerance fallback with full partial pivoting (`tol=1.0`), which also cured spurious far-field roots and a twin-T conjugate-pair stall. [Enhancement-171](#) had first fixed the KLU complex determinant itself (mixed real/complex pivot products and permutation parity — silent garbage for complex roots). [Enhancement-173](#)'s alternative root finder (`.options pzeig, shift-invert pencil` + its own Francis-QR eigensolver) is solver-agnostic by construction — the pencil is extracted densely through `SMPdenseExtractReal` from whichever solver holds the matrix — and returns identical roots under both.

[Enhancement-114](#) then fixes sensitivity under KLU. Sensitivity builds an auxiliary perturbation matrix `delta_Y` that is a plain Sparse matrix (it is only multiplied, never factored); two KLU setup blocks were gated on the *main* matrix's flag and dereferenced the NULL `delta_Y→SMPkluMatrix`, segfaulting on every DC/AC `.sens` deck. Gating them on `delta_Y`'s own flag keeps it Sparse under KLU too, so both DC and AC sensitivity now match Sparse exactly.

- Distortion (`.disto`) was Sparse-only — `distoan.c` had no KLU code, so under `.option klu` the (complex) distortion solves ran against a matrix left in real mode and silently produced no output. Surfaced while validating E-114. [Enhancement-115](#) fixes it: it converts the KLU matrix `real↔complex` around the distortion solve loop (exactly as `acan.c` does for AC), so `.disto` now matches Sparse bit-for-bit.

KLU discrepancies — all resolved

No example produces a different result under KLU than under Sparse. `KLU_XFAIL` is empty; the harness runs every example under both solvers and expects agreement. (Balanced-output pole-zero — formerly a genuine "not (yet) supported" *skip*, not a wrong result — runs under KLU since [Enhancement-172](#).)

`opamp741` — was the last one; the real cause was the integration method, not KLU. The transistor-level [opamp741 example](#) (a μ A741 from ~70 lines of Verilog-A BJT) used to abort under KLU on its large-signal slew test (`pulse(-5 5 ...)` into the follower): output-stage transistors switch off at the slewing edge, their transconductances collapse, KLU declares the Jacobian singular ($\times 1.01/02/b34/cm$), the timestep collapses, and ngspice aborted at $t \approx 2.03 \mu s$ — while Sparse ran the full `tran 20n 80u` cleanly.

The near-singular Jacobian at the edge was manufactured by the default trapezoidal integrator, which is non-dissipative and *rings* on the stiff feedback slew, driving the output transistors hard off. It is that ringing — not KLU's linear solve — that produced the collapse: Sparse's dynamic Markowitz re-ordering happened to survive the near-singular step where KLU's fixed symbolic ordering could not, but the step should never have been that singular. Switching the two transient decks to Gear (`.option method=gear`, dissipative BDF-2) damps the ringing so the edge is never near-singular: the slew now runs to completion under both solvers and the results agree to ~8 sig figs (final `V(out)` — 3.3153833712 KLU vs — 3.3153833649 Sparse), with every figure of merit (`Aol`, `fu`, `PM`, `slew rate`, `offset`, `swing`) identical between the solvers. `opamp741` was removed from `KLU_XFAIL` accordingly.

(This corrects the earlier reading of this case as an unfixable KLU linear-solver limitation needing a hybrid Sparse-fallback: the earlier pivoting/ordering knobs [E-116] left the *trapezoidal* run's abort unchanged because they addressed the symptom, not the ringing that caused it. The genuinely structural KLU issues were the two below, which E-116 did fix.)

Two former discrepancies, now fixed (E-116). `groundcontrib` (node-to-ground `V(p,gnd) <+ 1.5 read v(p)=0` under KLU instead of 1.5) and `hierbranch` (hierarchical branch-*current* probes read 0) were both structural, not numerical: an OSDI internal node that appears in no Jacobian entry — a ground reference whose branch contribution drops its column — was allocated its own all-zero solver row, which Sparse tolerates but KLU treats as structurally singular. [Enhancement-116](#) ties such decoupled internal nodes to ground at setup, so both now match Sparse under KLU.

Convergence aids and later solver fixes

convhelp (E-204) walks the standard DC continuation ladder automatically — plain Newton, then gmin stepping, then source stepping, then pseudo-transient — reporting which rung succeeded, instead of leaving the user to try each by hand.

DC false convergence (E-256) was a genuine wrong-answer bug: a B-source could report a spurious operating point that Newton accepted. The fix is a KCL merit test — if the residual current imbalance exceeds a threshold the “converged” point is rejected and the solver falls through to gmin/source stepping. Companion fixes cover the `.tran` operating point and `.dc` sweeps (E-257, E-258).

KLU glue hardening (E-232) fixed a null-check ordering bug and the collapse-map handling in `kclusmp.c`. Its follow-up (E-233) is worth reading for a negative result: a suspected `pz/sens` leak was investigated and refuted, and the collapse path was measured genuinely dead (0 of 191 corpus cases), rather than being “fixed” speculatively.

The dual-solver test harness

Every `verify_*.py` in `examples/` is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse / .option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS klu=PASS => OK ===
```

KLU_XFAIL is empty (opamp741, its last member, was resolved — see above), and since [Enhancement-172](#) there is no KLU-unsupported *analysis* left to auto-skip. A separate SPARSE_ONLY registry (`{highsigma, yield, cmc-sweep}`) marks examples whose KLU pass is not *wrong*, just slow — today those are the heavy Monte-Carlo batteries (thousands of runs each), which run Sparse-only by default (reported `klu=SKIP`). The periodic-steady-state examples (`rfpss`, `rfa` analyses) used to live in that registry too, because KLU re-factors every PSS shooting step and a 1024-sample `.pss` cost 10–15 min; [Enhancement-176](#)’s driven-mode shooting made them fractions of a second, so the whole RF suite now runs under both solvers on every sweep. Env escape hatches: `NGSPICE_SOLVER=klu|sparse` runs once under one solver; `NG_BOTH=0` disables the dual run; `NG_SLOW_KLU=1` forces the skipped heavy KLU passes back on.

Sweep result (175 verify scripts; 174 in the routine sweep, `cmcsweep` excluded for runtime): 174/174 OK — every example is `sparse=PASS` with `klu` in `{PASS, SKIP}`, i.e. the two solvers agree wherever KLU runs. (The `linesearch` example initially *crashed* under KLU, which [Enhancement-112](#) fixed; [Enhancement-113](#) then moved 10 examples from KLU-skipped to KLU-passing by enabling noise and single-ended pole-zero; [Enhancement-114](#) fixed the AC/DC-sensitivity crash under KLU, and — by fixing the deck-injector restore bug — un-masked two more XFAILs (`groundcontrib`, `hierbranch`); [Enhancement-115](#) then fixed distortion, so the `analyses` example now runs fully under KLU; and [Enhancement-116](#) fixed `groundcontrib` and `hierbranch` at their shared root cause; and `opamp741` — the last XFAIL — was resolved by running its stiff transient under Gear rather than the default trapezoidal method, so **KLU_XFAIL** is now empty. Later, [Enhancement-171/172](#) moved balanced-output pole-zero from auto-SKIP to passing, and [Enhancement-176](#) moved the heavy PSS examples from SPARSE_ONLY into the regular both-solver sweep.)

Conclusion: the two solvers agree across the entire suite — DC, AC, transient, noise, pole-zero (single-ended *and* balanced, plus `pzeig`), sensitivity, distortion, PSS, and the whole periodic small-signal family (KLU_XFAIL is empty; no analysis is Sparse-only since E-172, and no feature since E-180 — transient checkpoint/restart, the last holdout, now works under both solvers and even across them). Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.